INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 2

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica, trobarem un repàs i aprofundiment del conjunt d'instruccions del programa Ripes, el funcionament d'aquest i comprendre l'adreçament a memòria. Durant aquesta pràctica es farà servir el programa Ripes per programar els primers programes, fent ús de les diferents instruccions d'operacions aritmetic-lògiques, instruccions de salt, etc.

OBJECTIUS

L'objectiu d'aquesta pràctica és familiaritzar-nos amb el funcionament dels registres que hi ha a la CPU. Per tal d'assolir aquest objectiu, farem servir el simulador RIPES, en el que es realitzaran diferents programes per tal d'entendre com es comporten els registres i com es gestionen els adreçaments a memòria, juntament amb el tractament del contingut dels registres i com fer-los servir per tal de poder fer diferents salts condicionals i incondicionals a altres blocs de codi.

EXERCICI 1

Carrega el següent programa en el simulador RIPES:

```
 .data
Dades: .word 9, 3, 4, 1 # vector de 4 enters
Resultat: .word 0, 0 # vector de 2 enters
.text # directiva d'inici de programa
main:
la a0, Dades # a0 actuarà com a punter
lw a1, 0(a0) # 0(a0) = Dada[0]
lw a2, 4(a0) # 4(a0) = Dada[1]
lw a3, 8(a0) # 8(a0) = Dada[2]
lw a4, 12(a0) # 12(a0) = Dada[3]
loop:
add a5,a1,a2
```

```
add a6,a4,a2
addi a3, a3,-1
bgez a3, loop
sw a5, 16(a0) # 16(a0) = Resultat[0]
sw a6, 20(a0) # 20(a0) = Resultat[1]
end: nop #final de programa
```

Quina operació fa aquest codi?

Aquest programa guarda en dos vector diferents els diferents valors donats. Primer de tot fa un vector amb nombres que es sumaran posteriorment, i després fa un vector per emmagatzemar el resultat. En particular, veiem que en la posició de memòria 16+a0 es guardarà el valor de a5 i en 20 + a0 es guardarà el valor de a6.

Observem que en el programa el que s'està fent és sumar els valors de a1 i a2, que tenen assignats els valors de 9 i 3 respectivament, i es guarden al registre a5. Per altra banda, també es fa la suma dels valors de a4 i a2, que tenen assignats els valors de 1 i 3 respectivament, i es guarden al registre a6. Aquestes sumes es realitzen 5 vegades, ja que en el registre a3 es guarda el valor 4 del vector i aquest gràcies a la instrucció ADDI a3, a3, -1 , va decrementant el seu valor en 1.

Com BGEZ a3, loop fa que es torni a realitzar el loop mentre que el valor de a3 sigui major o igual a 0 (a3 >= 0), el comptador arribarà a 0 i es tornarà a realitzar la suma. Per tant, cada vegada que es fa el loop, es realitzaran les sumes dels registres i es guardarà el resultat (que cada vegada és el mateix) als registres corresponents.

Un cop acabat el programa quant val el contingut de A5?

Un cop s'acaba el programa, el valor de a5 correspon a la suma dels valors dels registres a1 +a2, suma que es va repetint en el loop però sempre amb el mateix valor. En particular, es fa 9 + 3 = 12, que és el contingut de a5 al final del programa.

Un cop acabat el programa quant val el contingut de A6?

Un cop s'acaba el programa, el valor de a6 correspon a la suma dels valors dels registres a4 +a2, suma que es va repetint en el loop però sempre amb el mateix valor. En particular, es fa 1 + 3 = 4, que és el contingut de a6 al final del programa.

Un cop acabat el programa quant val el contingut de A3?

El registre a3 fa la funció de comptador per tal de sortir del loop, amb el valor de 4. Degut a la instrucció BGEZ, una vegada el valor del registre a3 arribi a 0 (ja que degut a la instrucció ADDI a3, a3, -1 va decrementant el seu valor en 1), tornarà a repetir-se el loop, ja que BGEZ compara el valor de a3 amb el 0, tal que si a3 >=0, el loop continuarà. Per tant, tornarà a decrementar el seu valor i el contingut de a3 al final del programa serà -1.

Quines instruccions reals s'utilitzen per a les pseudoinstruccions nop i bgez? (mireu el codi màquina que es genera i consulteu la taula d'instruccions, utilitzant els camps opcode i func3/7 si cal).

Per part de nop, la instrucció que realment s'està utilitzant és addi x0 x0 0 segons el codi màquina, és a dir que el que s'està fent és sumar-li 0 a x0 i guardar-lo a x0, cosa que per defecte x0 sempre ha de tenir valor 0.

Per part de bgez, veiem que el que ens mostra el codi màquina és bge x13 x0 -12 <loop>. Això ens indica que està comparant el valor de la posició corresponent a x13, que és la del registre a3, amb el valor que hi ha a x0, que sempre és 0. Per tant, en realitat s'està fent servir un bge per comparar a3 amb x0. Per tant, veiem que la instrucció bgez ja inclou la comparació amb el 0. Es pot observar en el codi màquina que aquesta instrucció es troba en la posició 24h = 36d, i aquest -12 serveix per tornar a la posició 24d = 18h, on trobem l'inici del loop.

En quina posició de memòria es troben els valors de Resultat? En quina posició de

memòria es guarda el contingut del registre A6?

Els valors de Resultat es guarden en les posicions de memòria 16 + [a0] i 20 + [a0]. El contingut del registre 16 finalment es guarda en la posició 20 + [a0]

Quan executem bgez, quin registre es veu afectat?

Quan executem bgez, veiem que això és una comparació amb el zero, és a dir x0. En particular, en aquest programa s'està comparant amb a3, per tal de veure si el contingut de a3 és major o igual a 0. Per tant, el registre que es veu afectat és el a3.

EXERCICI 2

Feu un programa similar als de l'exercici 1. Inicialitzeu primer el contingut de A0. Carregueu al A1 el valor 0000110000001011b. Carregueu al registre A2 el valor 0000000000010001b. Feu l'operació A0<= A0+A1 i decrementeu el valor de A2. Feu això en un bucle fins que el valor contingut en A2 sigui 0.

Aquest és el codi del programa:

```
.data
Dades: .word 3083, 17
.text # directiva d'inici de programa
main:
add a0, zero , zero
la a3, Dades # a3 actuarà com a punter
lw a1, 0(a3)
lw a2, 4(a3)
loop:
add a0, a0, a1
addi a2, a2,-1
bgtz a2, loop
```

```
end:
nop #final de programa
```

Quina operació matemàtica està realitzant aquest codi?

El que està fent aquest codi és fer una multiplicació, ja que donat el valor de a1 i el valor de a2, el loop es fa 17 vegades, ja que per cada iteració, el valor de a2 decrementa en 1, cosa que provoca que com és 17 en decimal, s'hagi de decrementar 17 vegades per arribar a 0.

Per tant, 17 vegades es produeix l'operació a0 = a0 +a1. El que estem fent és, degut a que a0 té valor inicial 0, sumar 17 vegades el valor 3083 en decimal de a1, definició del que és una multiplicació.

Quant val el contingut d'A0, A1 i A2 quan acaba el programa?

En acabar el programa, veiem que a1 no ha estat modificat, per tant continua amb el valor 3083. Per altra banda, el valor de a2 al final del programa és 0, ja que l'hem decrementat 17 vegades perquè l'hem utilitzat com a iterador. Per últim, a0 contindrà el resultat que ens proporciona el codi, que és 52411, tal i com era d'esperar de la multiplicació.

Quantes iteracions del bucle executa el codi?

Tal i com s'ha dit prèviament, el registre a2 es fa servir com a iterador, per tant en decrementar en 1 el seu valor cada vegada, al cap de 17 iteracions, el programa sortirà del bucle, ja que a2 té valor 17 i arribarà a 0 després de decrementar 17 vegades.

EXERCICI 3

Aquest és el codi del programa:

```
.data
Dades: .word 3, 3
.text # directiva d'inici de programa
main:
        la a3, Dades # a3 actuarà com a punter
        lw a1, 0(a3)
        lw a2, 4(a3)
        bgt a1, a2, sum
        bgt a2, a1, diff
        beq a1, a2, mult
sum:
        add a0, a1, a2
        j end
diff:
        sub a4, a2, a1
        j end
mult:
        mul a0, a1, a2
        j end
end:
        sw a0, 36(x0)
        nop #final de programa
```

Expliqueu el funcionament del programa que heu implementat.

El programa comença guardant els dos valors donats en un vector, tenint el registre a3 com a punter. Seguidament, s'inicialitza el registre a0 amb la instrucció add a0, zero,

zero per emmagatzemar el resultat de l'operació i s'assignen aquests valors als registres a1 i a2 i posteriorment es fan ús de les instruccions bgt per comparar els dos nombres i saber quin és més gran i després beq per saber si son iguals.

Si es compleix el primer bgt, és a dir a1 > a2, saltarà a la posició de memòria sum, on es realitzarà la suma amb la instrucció add a0, a1, a2. Posteriorment, per evitar fer les altres línies de codi, s'utilitza la instrucció j end per fer un salt incondicional a la posició de memòria end, per acabar el programa.

En cas que el primer bgt no es compleixi i el segon sí, és a dir a2 > a1, es saltaria a la posició de memòria diff, on es realitzaria la resta a2-a1 amb la instrucció sub a0, a2, a1 i seguidament j end.

En el cas que no es compleixi cap dels dos, es complirà el beq, és saltarà a la posició de memòria mult, on es farà la multiplicació a1 · a2 amb la instrucció mul a0, a1, a2 i seguidament j end.

Finalment, una vegada està en la posició de memòria end, es guarda el contingut de a0 en la posició de memòria que s'ha demanat (24h bytes després de l'inici de la secció .data), tal que la instrucció seria sw a0, 36(x0).

D'aquesta forma, cada vegada que es s'inicialitzi el programa, només es farà una operació, en funció dels valors donats.

TEÒRIC-PRÀCTIC

Feu un programa en assemblador que carregui dos valors de una determinada posició de memòria en dos registres: a1 i a0, faci la suma i ho guardi en a2. Finalment guarda el resultat en memòria.

El codi del primer apartat és el següent:

```
.data
Dades: .word 3, 3
.text # directiva d’inici de programa
main:
la a3, Dades # a3 actuarà com a punter
lw a1, 0(a3)
lw a0, 4(a3)
add a2, a1, a0
sw a2, 8(a3)
```

Observem que és un programa bastant simple, que fa la suma del contingut dels registres a1 i a0 i els guarda en el registre a2. Finalment, el contingut del registre a2 es guarda en la posició de memòria 8+ [a3].

Feu un programa en assemblador que carregui dos valors de la memòria en dos registres. Si els valors són iguals, que faci el producte, guardant el resultat en la memòria. Si un és negatiu que faci la suma i la diferència, guardant el resultat en memòria. Finalment, si els dos són negatius que calculi el Ca2 de tots dos i els guardi en memòria.

El codi del segon apartat és el següent:

```
.data
Dades: .word -2, -4 # Valors d'exemple
.text # directiva d’inici de programa
main:
        la a3, Dades # a3 actuarà com a punter
        lw a1, 0(a3)
        lw a2, 4(a3)
        beq a1, a2,mult
        j diferents
mult:
```

```
        mul a0, a1, a2
        j end
diferents:
        bltz a1, possible
        j positiu
positiu:
        bgtz a2, end
        j possible2
possible:
        bltz a2, ca2
        add a0, a1, a2
        sub a6, a1, a2
        j end

possible2:
        add a0, a1, a2
        sub a6, a1, a2
        j end
ca2:
        not a1, a1
        addi a1, a1, 1
        not a2, a2
        addi a2, a2, 1
        sw a1, 8(a3)
        sw a2, 12(a3)
        j end
end:
        sw a0, 8(a3)
        sw a6, 12(a3)
        nop
end2:
        nop
```

En aquest programa cal tenir en compte que si els dos nombres són iguals, llavors és fa un salt condicional a mult i es fa la multiplicació, finalitzant el programa. A més a més, si són positius, llavors no es fa cap operació.

En aquest programa es controlen els casos quan siguin diferents els dos nombres. Si són diferents, es fa un salt incondicional a 'diferents'. Llavors, es fa la primera comprovació de si a1 és negatiu. En cas afirmatiu, es fa un salt a 'possible', i allà es fa una comprovació amb a2. Si també es negatiu, es fa un salt a 'ca2' i es fan el complements a 2 dels nombres i es guarden els valors en altres posicions de memòria i posteriorment es faria un salt incondicional a 'end2'. En cas de ser a2 positiu, allà es faria la suma i la diferència i després es faria el salt incondicional a 'end'.

En el cas de que el a1 sigui positiu, es fa un salt a 'positiu' i allà es fa una comprovació amb a2. Si és positiu, llavors es fa un salt a 'end', ja que els dos valors serien positius. En cas contrari, es faria un salt incondicional a possible2, i allà ja es farien la suma i diferència amb els valors i seguidament el salt incondicional a 'end'.

Finalment, una vegada el programa es troba en 'end', es guarden els resultats de la suma i la diferència en diferents posicions de memòria.

Com a comentari, cal destacar que les instruccions add i sub es faran sempre realitzant les operacions a1+ a2 i a1-a2

CONCLUSIONS

En aquesta pràctica hem pogut observar com es comporten els programes respecte els loops i les instruccions de salts condicionals i incondicionals. Gràcies a aquestes instruccions, podem controlar quan fer determinades accions en el programa, com per exemple una suma si es compleix certa condició, o una multiplicació.

S'ha tractat d'observar com funcionen els programes i entendre on es guarden els valors en les diferents posicions de memòria.
A més a més, s'ha treballat més a fons l'ús del programa Ripes, on s'han adquirit més coneixements de cara a programar diferents programes amb diferents tipus d'instruccions.

En definitiva, en aquesta pràctica s'ha treballat la millora en la comprensió del programa Ripes i el llenguatge assemblador.